

Dưới đây là **bản hướng dẫn logic để bạn nắm bắt toàn bộ nội dung tài liệu "Agent2Agent Protocol" trong vòng 1 giờ**, chia thành các phần theo lộ trình học hiệu quả từ tổng quan đến chi tiết:

🧠 0. Mục tiêu đọc tài liệu trong 1 giờ

Bạn cần hiểu:

- Tại sao có vấn đề hợp tác đa agent?
- A2A Protocol giải quyết vấn đề đó như thế nào?
- Cấu trúc của A2A gồm những gì?
- Làm sao để xây dựng hệ thống multi-agent sử dụng A2A?

1 Tổng quan vấn đề và lý do ra đời A2A Protocol (10 phút)

🧩 Bối cảnh vấn đề

- Mỗi agent (LangGraph, CrewAI, LlamaIndex...) có cách triển khai khác nhau.
- Khi cần hợp tác xử lý quy trình phức tạp → cần giao tiếp liên agent.
- Vấn đề:** Không có giao thức chung → lỗi dây chuyền, khó mở rộng, bảo trì kém.

💡 Giải pháp: A2A Protocol

- Là một **giao thức chuẩn** cho phép các agent dị nền tảng giao tiếp với nhau một cách:
 - Hiệu quả** (giao tiếp tiêu chuẩn)
 - Linh hoạt** (mỗi agent có thể thay thế dễ dàng)
 - An toàn** (xác thực, phân quyền)
 - Có thể mở rộng** (thêm agent mới không ảnh hưởng hệ thống)

2 Cấu trúc và thành phần cốt lõi của A2A (15 phút)

📄 Agent Card

- JSON mô tả năng lực của Agent (`/.well-known/agent.json`)
- Bao gồm tên, URL, version, các skill, định dạng input/output, xác thực, v.v.

🔌 JSON-RPC 2.0

- Giao thức nền tảng cho A2A (các method: `tasks/send`, `tasks/get`, `tasks/cancel`, ...)
- Giúp client gửi task, nhận kết quả, theo dõi tiến trình, hủy task.

🧠 Các thực thể chính trong A2A

Thành phần	Mô tả
Task	Đơn vị công việc
Message	Một lượt hội thoại

Thành phần	Mô tả
Part	Một phần nhỏ (text, file, JSON...)
Artifact	Kết quả sinh ra từ task
A2A Client	Điều phối gửi yêu cầu
A2A Server	Xử lý và trả kết quả

3 Cách Agent giao tiếp với nhau qua A2A (10 phút)

🔄 Mô hình 2 agent tương tác:

1. **Discovery:** Gọi `.well-known/agent.json` để biết năng lực.
2. **Task Assignment:** Gửi task bằng `tasks/send`
3. **Communication:** Trao đổi qua Message/Part.
4. **Task Progress:** SSE/Webhook cập nhật trạng thái.
5. **Completion:** Trả về kết quả cuối cùng (text, file, URI...)

🔗 Mô hình nhiều agent:

- Mỗi agent là một A2A Server
- **Host Agent** là trung tâm điều phối (Routing Agent)
- Kết nối đến nhiều remote agents thông qua A2AClient và AgentCard

4 Cách xây dựng hệ thống sử dụng A2A (20 phút)

🏗️ Kiến trúc tổng thể

- **Gradio UI** → gửi truy vấn → **Host Agent**
- Host Agent phân tích → gửi task tới **Remote Agent**
- Remote Agent xử lý bằng LLM + MCP Tool → trả kết quả qua A2A → Host Agent tổng hợp trả lại UI

⚙️ Thành phần kỹ thuật

✅ **A2A Server (Remote Agent):**

- Xây dựng bằng LangGraph hoặc Google ADK
- Có class như `AirbnbAgent`, `WeatherAgent`
- Dùng toolset từ MCP để xử lý nghiệp vụ

✅ **A2A Client (Host Agent):**

- Sử dụng `RemoteAgentConnection` để kết nối đến các remote agent.
- Dùng `send_message()` để truyền task theo JSON-RPC
- Được khởi tạo từ `RoutingAgent`, sử dụng mô hình ngôn ngữ `gemini-2.5-flash`

5 Chạy thử demo A2A (5 phút)

 Github Repo:

<https://github.com/quaghien/A2A-samples>

 Chuẩn bị môi trường:

- Python 3.13, Node.js 20, `.env`, uv tool
- Clone repo, cài đặt, chạy Gradio UI

6 Tổng kết (Dành 5 phút cuối ôn lại)

- A2A giải quyết vấn đề chuẩn hóa giao tiếp đa agent
 - Giao thức chuẩn là JSON-RPC 2.0
 - Có Host Agent điều phối + Remote Agent xử lý chuyên biệt
 - AgentCard giúp discovery → client không cần biết logic nội bộ
 - Cấu trúc rõ ràng, mở rộng dễ dàng, hỗ trợ stream và bảo mật
-